Defense: likely questions and grounded answers

Twenty-eight questions a reviewer is likely to ask, with the answer and the file, command or figure that backs it. Nothing here is asserted without a source. The twenty-eight were re-checked against the repository at the final revision; every path and test name below resolves, and the figures match docs/METRICS.md. A shorter list for a judge who wants the essentials first is docs/FAQ_FOR_JUDGES.md.

1. How do you keep an observation separate from a hypothesis?

An observation is a byte range the engine read, with its packet, sequence and time. A hypothesis is a field the rule marks hypothesis: true, meaning the meaning of the bytes is assumed. The rule engine reports a value as a fact (value plus provenance); the alternatives engine reports a meaning as a candidate with a score. The two never share a code path. Evidence: src/protocol/engine.py, src/hypothesis/alternatives.py, REPORT.md section 3.

2. Why is the framing length-prefixed with `length_covers: payload`?

Because it is the only of three candidates that frames every clean stream exactly, with no leftover bytes and no truncated message. payload_and_length_field consumes the stream but leaves impossible command bytes; entire_message stops after the first size. Evidence: framing census in docs/CORPUS_ANALYSIS.md section 2.

3. Why did you widen the command set from `[1]` to `[1, 2, 3]`?

Because v1 produced eighty counterexamples, all "value 2 or 3 not in expected [1]", and the alternatives engine says the field is a small closed enum (entropy_enum: support 140, contradict 0) while constant (the v1 reading) is contradicted by 80 of 140. Evidence: docs/CORPUS_ANALYSIS.md sections 4-5, docs/RULE_V1_V2.md section 3.

4. Why is the command still a hypothesis after the check?

A corpus check confirms agreement, not meaning. 0x01/0x02/0x03 fit a command code, but the corpus is synthetic and small, so the rule keeps the flag. The field is confirmed in scope, not proven. Evidence: REPORT.md section 3, src/hypothesis/status.py.

5. What exactly is a counterexample?

A mismatched message, bound to its bytes and its source packet: session_id, direction, message_offset, message_length, bytes_hex, reason. It is not a discarded row; it is stored in VerificationReport.contradictions and shown in the GUI. Example: 02000003130055 in s2 A_to_B, reason "value 2 not in expected [1]". Evidence: src/hypothesis/corpus.py, docs/CORPUS_ANALYSIS.md section 4.

6. How are precision and coverage defined?

Precision is matched / (matched + mismatched), the share of messages the rule spoke about that it got right. Coverage counts how much of the corpus the rule addresses at all; streams outside the scope are not_applicable, not silently counted as matched. For v1 on corpus_capture_01: precision 60/140 = 0.43; v2: 140/140 = 1.00. Evidence: src/hypothesis/metrics.py, docs/CORPUS_ANALYSIS.md section 6.

7. Why is `B_to_A` reported as not_applicable?

The rule scopes to `A_to_B`; it makes no claim about the other direction. The engine records that as `not_applicable` rather than inventing a match. Evidence: `tests/protocol/test_engine.py::test_scoped_rule_marks_other_direction_not_applicable`.

8. What happens to a result produced by an older rule version?

It is marked `outdated`, with reason `superseded_by_newer_rule`. It stays on disk with its version, and the version comparison is the only place v1 and v2 numbers appear side by side. An old result is never shown as current. Evidence: `src/hypothesis/versioning.py`, `src/ui/model.py::_mark_outdated`, `tests/hypothesis/test_versioning.py`.

9. Why are gaps not zero-filled?

Zero-filling would invent bytes that were never observed and could turn an incomplete message into an apparently valid one. A gap becomes a `gap` diagnostic; a message over it is `incomplete`. Evidence: `tests/protocol/test_engine.py::test_gap_in_field_range_is_incomplete`, `docs/CORPUS_ANALYSIS.md` section 1.

10. What about an ambiguous overlap?

A conflicting retransmission keeps the first bytes and records an `ambiguity` diagnostic with both versions. A field over the ambiguity is `ambiguous`, not `matched`. Evidence: `tests/protocol/test_engine.py::test_ambiguity_in_field_range_is_ambiguous`.

11. Where does the rule stop?

At `synthetic_live`. v2 matches nothing there. The capture uses a little-endian length, so under the rule's big-endian length both streams are truncated: neither frames cleanly, and the scoped rule reports one `incomplete` message with the rest `not_applicable`. The rule fails on the length field before any command byte is judged, and it says so rather than guessing. Evidence: `docs/CORPUS_ANALYSIS.md` section 7, `tests/integration/test_reference_report.py::test_v2_fails_at_the_synthetic_boundary`.

12. Is "matched everywhere on the corpus" the same as proof?

No. It means the rule is confirmed inside the checked bytes. The whole point of the alternatives engine and the boundary capture is to keep that distinction visible. Evidence: `REPORT.md` section 3, section 8.

13. Why do alternative readings come from a fixed set?

The engine enumerates named candidates: `constant`, `counter`, `message_length`, `remaining_length`, `low_cardinality`, `alias_of_<field>`, the entropy bands (`entropy_enum`, `entropy_parameter`, `entropy_random`), `periodicity`, `bit_pattern_<width>`, `flags_high`, `delta_correlation`, `delta_encoding`, `endianness`, `offset_shift_+1`/`offset_shift_-1`, `xor_mask_<mask>`, `checksum_<algorithm>_<region>` and `journal_<key>`. A meaning outside the set will not be proposed. This is stated as a limitation on the deck and here. Evidence: `src/hypothesis/alternatives.py`, `docs/PRESENTATION_VERIFICATION.md`, the limitations slide.

14. How do you know the corpus rule is not overfit to one capture?

v2 is verified on a second, independent capture (`corpus_capture_02`) without a single counter-example: 40 matched, 0 mismatched. Evidence: `tests/integration/test_reference_report.py::test_v2_transfers_to_the_second_capture_without_counterexamples`.

15. Why separate `rule_id` from `rule_version`?

`rule_id` names the rule across its history; `rule_version` counts changes. Reloading identical text is a no-op, so a re-run does not mark its own result outdated, while any change bumps the version. Evidence: `src/protocol/rule.py`, `src/ui/model.py::RuleModel.reload`.

16. How is a rule validated?

Against `docs/schemas/rule.schema.json` (JSON Schema draft 2020-12) with `jsonschema.Draft202012Validator`, and by the parser in `src/protocol/rule.py`. Invalid framing or a bad field offset is rejected before the rule runs. Evidence: `tests/protocol/test_rule.py`.

17. Why not PyShark or Kaitai for the reading itself?

PyShark needs `tshark`; the tool must run with only Python dependencies. dpkt reads both pcap and pcapng with no external binary. Kaitai is offered as an export of a finished rule, not as the engine. Evidence: `ARCHITECTURE.md`, `src/protocol/export.py`.

18. How does the tool behave with no corpus?

The corpus smoke tests skip rather than fake a result, and the CLI fails with a clear error if a file is missing. Nothing is invented when the data is absent. Evidence: `tests/integration/test_reference_report.py` (skip marker), `docs/demo_cli.md`.

19. Why does the flags byte have no meaning stated?

It is constant (`0x00`) in every observed message, so no reading can be told apart from any other; the rule declares `expected: [0]` and nothing more. A meaning would be a guess with no evidence. Evidence: `docs/CORPUS_ANALYSIS.md` section 3, `docs/PRESENTATION_VERIFICATION.md`, the limitations slide.

20. What would you do next?

Three concrete steps, each aimed at a limitation above: capture real device traffic to test the synthetic-corpus caveat; parse the `B_to_A` responses by field so the rule covers both directions; and drive a parameter across sessions to see whether the flags byte ever changes, which would let a second reading be distinguished. Evidence: `REPORT.md` section 12, deck limitations slide.

21. Why not use Wireshark as the backend instead of dpkt?

Wireshark is a program, not a library; using it would mean shelling out to `tshark`, so the tool would depend on an external binary and its version. dpkt reads both pcap and pcapng in pure Python with no external process, which keeps the project reproducible from `pip install` alone. The capture module is a thin layer over dpkt and is tested on both container formats. Evidence: `ARCHITECTURE.md` (dependency table), `src/capture/`.

22. How do you know the rule is not overfit to the corpus?

Overfitting is tested by transfer, not by the fit itself. Rule v2 was fitted on `corpus_capture_01` and then applied unchanged to `corpus_capture_02`, a separate record: 40 matched, 0 counterexamples. It was also applied to `synthetic_live`, where it matches nothing; a rule tuned to the corpus would have no reason to fail there, and the tool records that failure instead of hiding it. Evidence: `REPORT.md` sections 7-8, `tests/integration/test_reference_report.py::test_v2_transfers_to_the_second_capture_without_counterexamples`.

23. The corpus is small; is that a problem?

The corpus holds 185 journal transactions and 369 framed messages across three captures (280 in `corpus_capture_01`, 80 in `corpus_capture_02`, 9 in the defects capture). It is enough to show that the framing is exhaustive and that the command set is closed on the observed values, but not enough to prove a general grammar. The claim is kept to what the bytes support: the field is confirmed in scope, and the rule keeps `hypothesis: true` where the meaning is inferred. Evidence: `REPORT.md` section 3, section 12, `docs/CORPUS_ANALYSIS.md` section 8.

24. How do you check the rule is not fitted to one file?

Two ways. First, transfer: the same rule is run on a second capture it was never fitted to, and the counters are checked there. Second, the counterexamples: v1 is kept with its eighty contradictions, so the boundary of the rule is visible as data, not asserted. A rule that only worked on the file it was written against would show zero counterexamples on that file and a wall of them on the next, which is exactly what the boundary capture demonstrates. Evidence: `REPORT.md` sections 5, 7, 8.

25. Where are the researcher's observations stored?

Observations and hypotheses live in the rule JSON and in the structured research report. Each field carries its offset, type and `hypothesis` flag, and every verdict carries the byte range and the packet it came from, so the reasoning is reconstructable from the documents alone. A separate `annotations.sqlite` table is not implemented; that is a deliberate choice in favour of a self-contained JSON format that moves with the project and adds no dependency. The task statement does not require a particular storage mechanism. Evidence: `docs/schemas/rule.schema.json`, `REPORT.md`, `src/hypothesis/corpus.py`.

26. How would the check be reproduced a year later on another machine?

Every number is produced from the shipped files by a command, and the environment is pinned: Python 3.12 or newer and the dependencies in `pyproject.toml`. The report states how each figure is generated, so it can be regenerated rather than trusted. If a corpus file is missing, the smoke tests skip and the CLI fails with a clear error instead of inventing a result. Evidence: `docs/PORTABILITY.md`, `docs/demo_cli.md`, `pyproject.toml`.

27. How do you know `length_covers: payload` is the right semantics?

It is not asserted; it is the only of three candidates that frames every clean stream with no leftover bytes and no truncated message. `payload_and_length_field` consumes the stream but leaves command bytes at impossible values, and `entire_message` stops after the first size. The decision is a measurement on the corpus, fixed in both rule versions as `"length_covers": "payload"`. Evidence: `docs/CORPUS_ANALYSIS.md` section 2, `REPORT.md` section 4.

28. Why a GUI at all, if everything runs from the CLI?

The CLI is the primary, scriptable path: apply, verify, diff, metrics, correlate and export all run without a window. The window exists for the part a terminal is bad at: seeing bytes, provenance and counter-examples together, and moving between a message, its packet and its verdict. Both paths call the same engine, so the window never computes a second answer. Evidence: `docs/demo.md` (window walkthrough), `docs/demo_cli.md` (the same investigation without the window), `src/ui/`.

Where the numbers come from

```
python -m pytest tests/integration/test_reference_report.py -q
python -m src.protocol.cli verify --rule examples/corpus_rule_v1.json \
    --capture tests/corpus/reference_export/corpus_capture_01.normalized.json --out /tmp/v1.json
python -m src.protocol.cli verify --rule examples/corpus_rule_v2.json \
    --capture tests/corpus/reference_export/corpus_capture_01.normalized.json --out /tmp/v2.json
python -m src.protocol.cli diff --rule-a examples/corpus_rule_v1.json \
    --rule-b examples/corpus_rule_v2.json --report-a /tmp/v1.json --report-b /tmp/v2.json
```